

Linked List



Dr. Md. Atiqur Rahman
Assistant Professor
Dept. of Computer Science & Engineering

Linked List

A **linked list (also called self referential)** is a linear data structure that consists of a sequence of elements, where each element points to the next element in the sequence. Unlike arrays, linked lists do not have a fixed size, and the memory for elements is allocated dynamically.

Each element in a linked list is called a "node," and each node has two components:

- ❖ **Data:** The actual data or value stored in the node.
- ❖ **Next (or Link):** A reference or pointer to the next node in the sequence.

The last node in a linked list typically has a reference of null or None to signify the end of the list.

There are various types of linked lists, including:

1. **Singly Linked List:** Each node points only to the next node in the sequence.
2. **Doubly Linked List:** Each node points to both the next and the previous nodes, allowing for traversal in both directions.
3. **Circular Linked List:** The last node in the list points back to the first node, forming a loop.

Linked List

Q. What are the advantages of linked list over array?

Dynamic Size:

- ❖ **Linked List:** Linked lists can dynamically grow or shrink in size without the need to predefine the size. Elements can be easily added or removed from any position in the list.
- ❖ **Array:** Arrays have a fixed size, and resizing them often involves creating a new array and copying elements, which can be inefficient.

Efficient Insertions and Deletions:

- ❖ **Linked List:** Insertions and deletions at any position in the linked list are more efficient since it only involves updating pointers. No need to shift or resize elements.
- ❖ **Array:** Insertions or deletions in the middle of an array require shifting elements to accommodate the change, which can be time-consuming.

Memory Utilization:

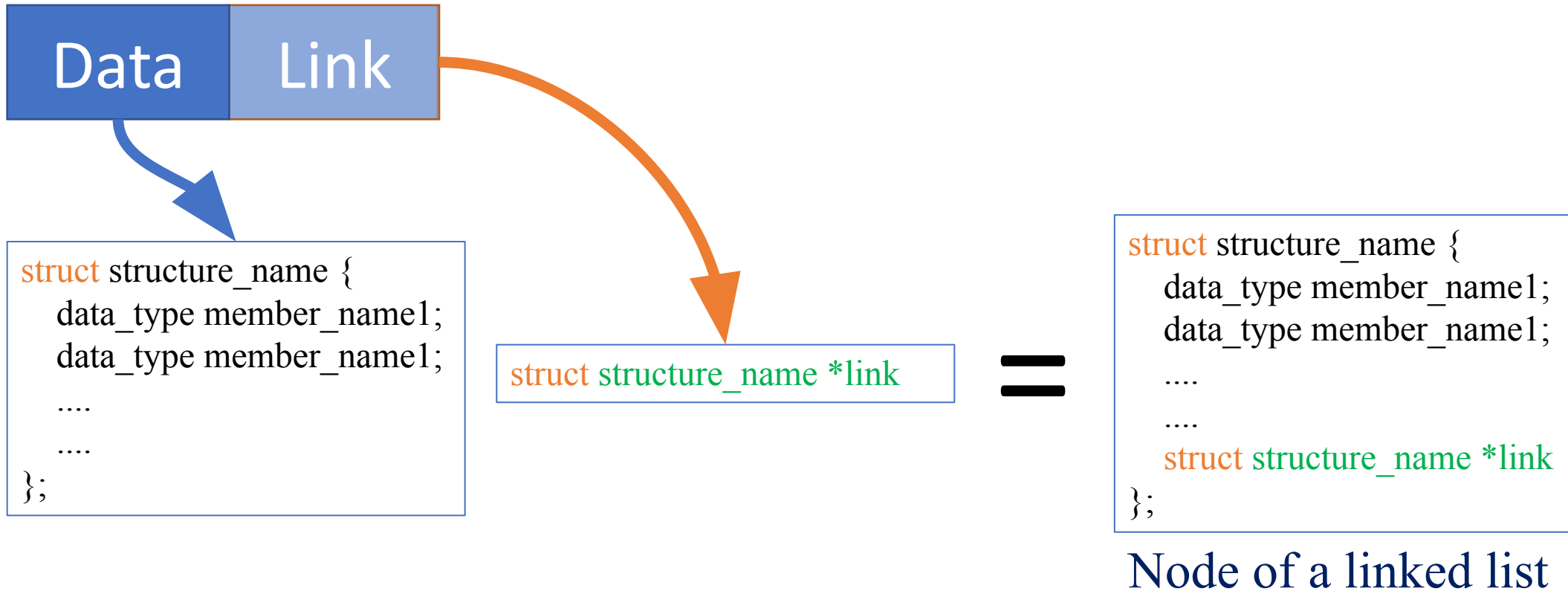
- ❖ **Linked List:** Memory is allocated dynamically for each node, allowing efficient utilization of memory. It avoids the need for a fixed-size contiguous block of memory.
- ❖ **Array:** Requires a contiguous block of memory with a fixed size, leading to potential wasted memory if not fully utilized.

No Need for Pre-allocation:

- ❖ **Linked List:** Elements can be added without the need to pre-allocate a specific size, making it suitable for scenarios with unpredictable data sizes.
- ❖ **Array:** Requires pre-allocating a specific size, and resizing might be necessary if the size is exceeded.

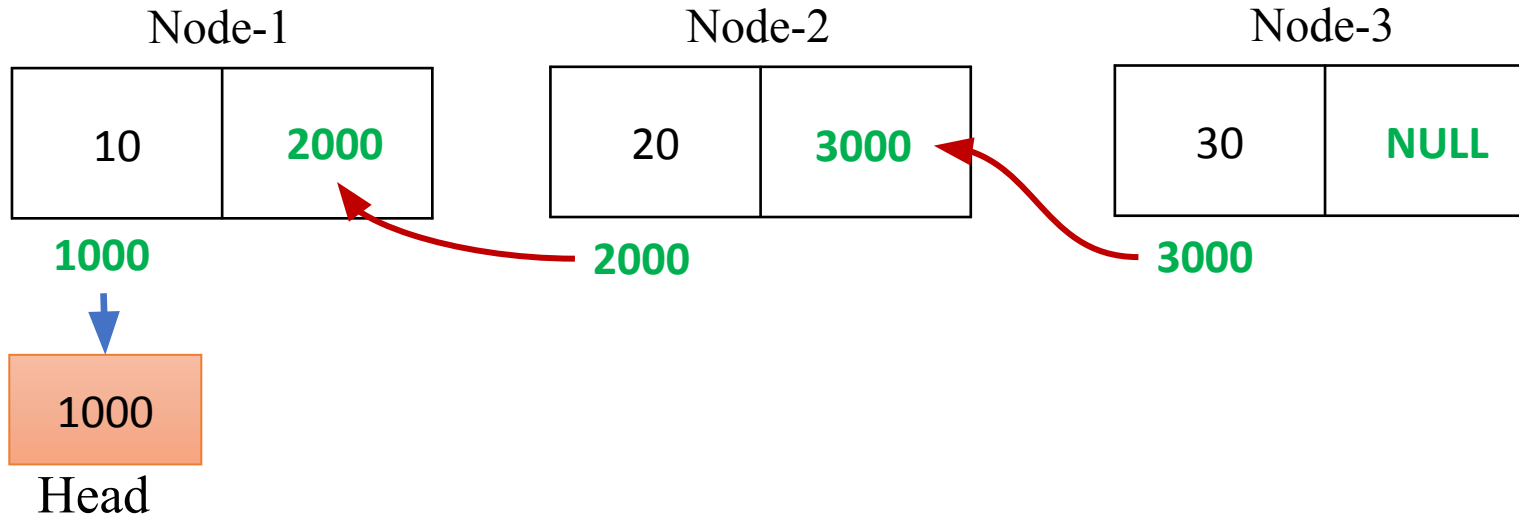
Linked List

Node



Linked List

Suppose we want to store a list of numbers in a linked list, and the numbers are 10, 20, 30



```
struct numbers {  
    int a;  
    struct numbers *link  
};
```

Declaration

This is called a Single Linked List

Linked List

Array:

10	20	30
1000	1004	1008

Creating a single Linked list

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    int a;
    struct node *link;
};

int main()
{
    struct node * firstNode =NULL;
    firstNode =malloc(sizeof(struct node));
    firstNode ->a=10;
    firstNode ->link=NULL;
    printf("%d", firstNode ->a);
    return 0;
}
```

Linked List

Creating a single Linked list

(Add second node)

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    int a;
    struct node *link;
};

int main()
{
    struct node * firstNode =NULL;
    firstNode =malloc(sizeof(struct node));
    firstNode ->a=10;
    firstNode ->link=NULL;
    printf("%d ", firstNode->a);
```

```
// Creating next node
    struct node *secondNode=malloc(sizeof(struct node));
    secondNode->a=20;
    secondNode->link=NULL;
    printf("%d", secondNode->a);

    // Connecting the secondNode with the first node (head)
    firstNode ->link = secondNode;

    return 0;
}
```

Linked List

Creating a single Linked list

(Add third node)

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    int a;
    struct node *link;
};

int main()
{
    struct node *firstNode=NULL;
    firstNode =malloc(sizeof(struct node));
    firstNode ->a=10;
    firstNode ->link=NULL;
    printf("%d ", firstNode ->a);
```

```
// Creating second node
    struct node *secondNode=malloc(sizeof(struct node));
    secondNode->a=20;
    secondNode->link=NULL;
    printf("%d ", secondNode->a);
    // Connecting the secondNode with the first node (head)
    firstNode ->link = secondNode;

    // Creating third node
    struct node *thirdNode=malloc(sizeof(struct node));
    thirdNode->a=30;
    thirdNode->link=NULL;
    printf("%d ", thirdNode->a);
    // Connecting the thirNode with the second node (secondNode)

    // Two ways for connecting
    secondNode->link = thirdNode;
    // firstNode ->link->link= thirdNode;

    return 0;
}
```


Linked List

Creates a linked list for N elements and prints the elements

```
#include <stdio.h>
#include <stdlib.h>
struct node{
    int a;
    struct node *link; };
int main(){
    int N;
    scanf("%d",&N);
    struct node *head=NULL;
    struct node *current=NULL;
    for(int i=0;i<N;i++){
        #Creating Node
        struct node *NewNode=malloc(sizeof(struct node));
        NewNode->a=rand()%100; //scanf("%d",&NewNode->a);
        NewNode->link=NULL;
        #Creating Linkedlist
        if(head==NULL) {
            head=NewNode;
            current=head; }
        else {
            current->link = NewNode;
            current=NewNode; }
    }
```

```
        #Traversing the linked list
struct node *currentNode;
currentNode=head;
while(currentNode!=NULL)
{
    printf("%d ", currentNode->a);
    currentNode=currentNode->link;
}

return 0;
}
```

Linked List

COUNTING THE ELEMENTS

```
void count_of_nodes(struct node *head) {  
    int count = 0;  
    if(head == NULL)  
        printf("Linked List is empty");  
    struct node *ptr = NULL;  
    ptr = head;  
    while(ptr != NULL) {  
        count++;  
        ptr = ptr->link;  
    }  
    printf("%d", count);  
}
```

PRINTING THE DATA

```
void print_data(struct node *head) {  
    if(head == NULL)  
        printf("Linked List is empty");  
    struct node *ptr = NULL;  
    ptr = head;  
    while(ptr != NULL) {  
        printf("%d ", ptr->data);  
        ptr = ptr->link;  
    }  
}
```

Array

COUNTING THE ELEMENTS

```
#include <stdio.h>

int main() {
    int arr[] = {45, 98, 3};
    int n;
    n = sizeof(arr)/sizeof(int);
    printf("%d", n);
    return 0;
}
```

PRINTING THE DATA

```
#include <stdio.h>

int main() {
    int arr[] = {45, 98, 3};
    int n, i;
    n = sizeof(arr)/sizeof(int);
    for(i=0; i<n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

LINKED LIST

Counting the elements: $O(n)$

Printing the data: $O(n)$



ARRAY

Counting the elements: $O(1)$

Printing the data: $O(n)$

Thank You